

ResilienceOps

Infrastructure Fault Testing and Recovery

Technical Documentation

Ritvik Indupuri

1 Project overview

ResilienceOps, published in the InfraAttack repository, exercises a containerized service stack with real HTTP traffic and deliberate resource or network faults. A Python orchestrator coordinates fault planning, execution, health checks, rule-based diagnosis, recovery and incident reporting. Claude supplies campaign descriptions, fault explanations and supplementary root-cause analysis.

The target application consists of a FastAPI gateway, order service and payment service. Redis is used for short-lived order caching. Docker Compose brings these services together with traffic generation, Prometheus, Grafana and an agent container. Kubernetes manifests define target workloads and a dedicated agent sandbox; Terraform provisions an AWS host and network resources.

Subsystem	Implementation
Traffic and application	Asynchronous HTTP sessions, gateway routing, in-memory orders, Redis caching and generated payment responses.
Red Team	Attack planner plus resource and network execution roles.
Blue Team	Health monitor, investigator and recovery fixer; a separate report generator writes Markdown.
Observability	Prometheus metrics, Grafana dashboards, local JSONL events and best-effort CloudWatch logging.
Infrastructure	AWS EC2 with Docker and k3s bootstrap; Kubernetes and Compose deployment definitions.

Review scope

This document follows orchestrator.py and the modules it imports. It distinguishes executed Python behavior from model descriptions and deployment intent. The alternate arena_runner.py and older attacker/defender modules form a separate execution path; they are not combined into the primary six-role cycle.

Agent model

Three Red Team classes and three Blue Team classes are created per cycle, with PostMortemGenerator as an additional support component. Four roles can call Claude: planner, resource stresser, network injector and investigator. Only one of the two fault executors runs in a cycle. The monitor, recovery fixer and report generator do not call an LLM in this path.

2 System architecture

The traffic generator issues requests to the gateway. The gateway forwards `/api/orders` paths to order-service and `/api/payments` paths to payment-service, stripping those prefixes. It uses a ten-second upstream HTTP timeout and returns 504 for timeout or 502 for other upstream failures.

Orders are stored in a Python dictionary and cached in Redis for 300 seconds. PostgreSQL 16 is declared in Compose and connection variables are supplied, but the order implementation contains no PostgreSQL client or database writes. Payment responses generate transaction identifiers locally; no external payment-provider authorization is performed.

Sandbox execution

The documented AWS operating model launches the orchestrator inside `sre-agent-sandbox`. This is the agent execution boundary in the architecture. The sandbox manifest defines `red-team-sandbox` and `blue-team-operator` deployments; the orchestrator itself initializes both teams in one Python process. Deployment names therefore do not imply six independent running agent pods.

The sandbox containers initially install dependencies and wait. Project source, credentials and execution must be supplied separately. Compose defines an `agent-sandbox` image that runs five cycles with ten-second delays and mounts reports back to the host.

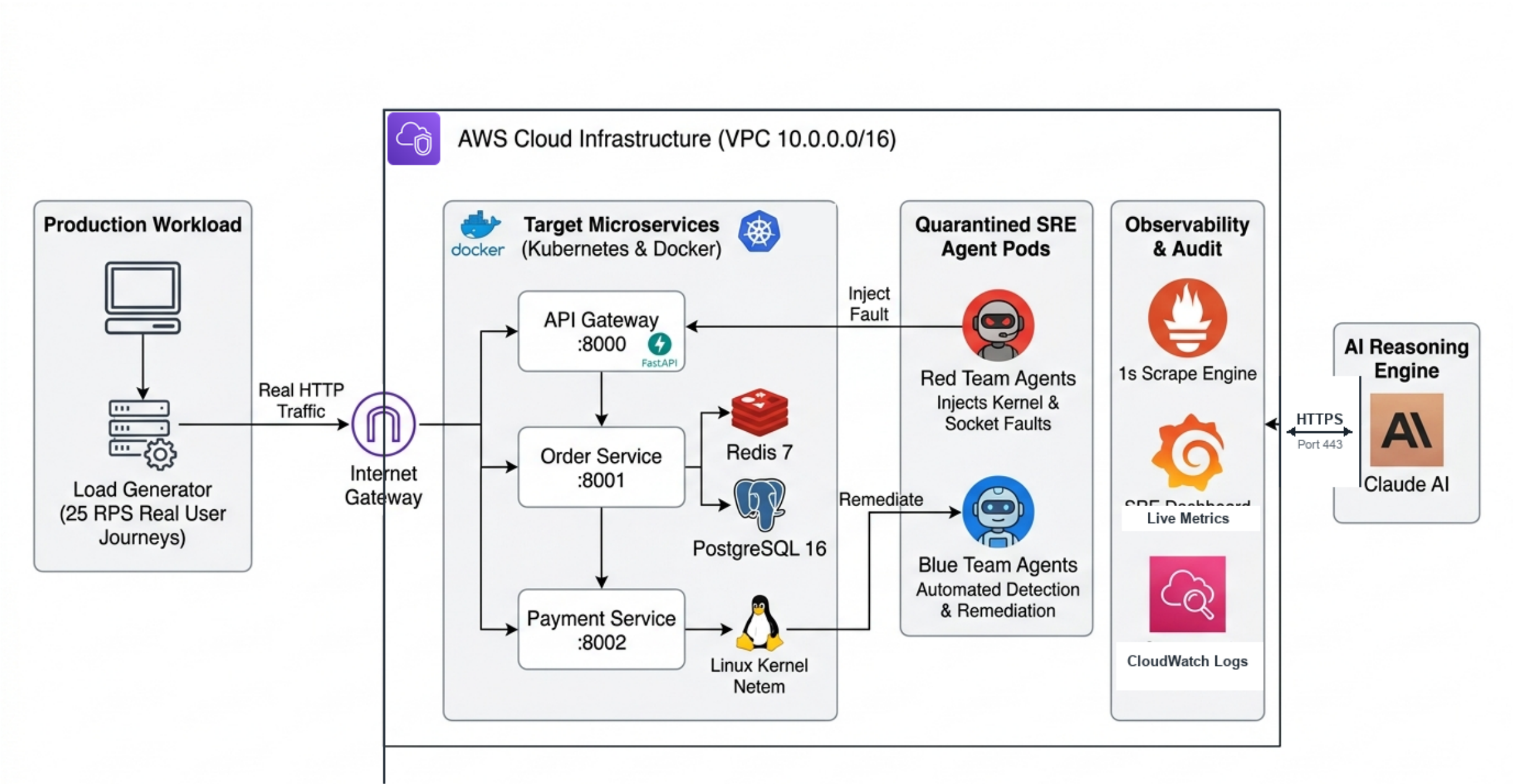
Telemetry and control

Prometheus scrapes `/metrics` on all three application services. Grafana reads Prometheus for display. The incident monitor independently probes `/health` endpoints; it does not query Prometheus in the primary cycle. Faults and resets are HTTP calls to the order/payment services. Kubernetes tools supplement diagnosis and remediation when a usable `kubectl` environment is present.

Architecture figure

The following figure reproduces the README architecture image unchanged. It presents the broader platform layout. In the active code, the gateway routes independently to order and payment services, and order persistence uses memory and Redis. The PostgreSQL link in the overview is not an implemented database-write path. The Terraform deployment uses one EC2 host, as detailed in Section 8.

System architecture



Original README architecture overview. See Section 2 for active service routing and storage behavior. Deployment boundaries and the single-host Terraform topology are explained in Sections 7 and 8.

2.1 System architecture walkthrough

Read the preceding figure as a platform overview. The flow below describes the implemented request, control and telemetry paths; the infrastructure boundaries are explained in Sections 7 and 8.

1. Generate application traffic

The traffic generator sends real HTTP sessions to the API gateway: a health check, order creation, payment submission when an order ID is available, and an order-history request. Its configured rate controls sessions; a session can contain several HTTP requests.

2. Route requests to the target services

The gateway forwards order and payment paths to their respective services. These are independent upstream routes: the order service does not forward requests to the payment service. The workload generator initiates the payment request through the gateway.

3. Maintain application state

The order service keeps orders in memory and caches them in Redis for 300 seconds. The payment service generates transaction responses locally. PostgreSQL appears in the README overview and Compose configuration, but the active order code does not write to it.

4. Apply faults and recovery actions

The orchestrator initializes both teams within its Python process in the agent execution environment. Executors call the services' fault endpoints; the recovery role calls reset endpoints and can invoke Kubernetes actions. Resource faults target the order service, while network faults use the payment service's tc endpoints.

5. Read metrics and detect incidents

Prometheus scrapes application metrics and Grafana displays them. Separately, the incident monitor probes gateway, order and payment health directly. Prometheus is therefore the dashboard metrics source, not the source queried by the primary incident-detection loop.

6. Supply model context and record evidence

The planner, selected executor and investigator can request Claude text through the shared engine. Python retains control of fault selection and recovery rules. Structured events are written locally, posted to the gateway and sent to CloudWatch on a best-effort basis; the report writer produces a Markdown incident report.

Source paths: traffic-generator/, services/*/main.py, orchestrator.py and agents/common/. The README figure is retained unchanged; the walkthrough states the implemented relationships.

3 Traffic and application interfaces

Real request workload

The traffic generator performs a health request, creates an order, submits a payment when an order ID is returned, then reads recent orders. It uses an async queue of 100 items, shared HTTPX connection limits of 20 keep-alive and 50 total connections, and configurable workers. Requests reach the actual HTTP services.

RPS_TARGET controls session dispatch, not an exact count of HTTP requests per second. Each session may issue four requests. Source defaults are 25 sessions per second and ten workers; Compose supplies 20 and eight. Queue pressure, latency and timeouts affect achieved throughput. A five-second reporter prints cumulative outcomes and latency percentiles over the latest 1,000 successful response measurements.

Service / route	Behavior
Gateway /health	Reports the gateway itself as healthy; does not aggregate dependency health.
Gateway /api/orders{path}	Forwards GET, POST, PUT and DELETE to the order service.
Gateway /api/payments{path}	Forwards the same methods to the payment service.
Gateway /api/agent-events	Accepts structured event summaries and increments a labeled metric.
Order GET /orders	Returns the latest 20 in-memory order values.
Order POST /orders	Creates an order and attempts Redis caching; connection exhaustion can produce HTTP 503.
Payment POST /payments	Returns a generated settled transaction unless outage state is active.
All services /metrics	Exposes Prometheus application and process metrics.

Health behavior

Order health reports Redis status in its JSON body but can still return HTTP 200 when Redis is unreachable. Payment health reports network-rule state and returns 503 only when its outage flag is enabled; kernel delays can instead cause probe timeouts. These response contracts determine what the monitor can infer.

4 Agent cycle and model calls

A cycle takes a baseline snapshot, asks the planner for a short campaign narrative, dispatches one executor and checks for an incident. An explicit `--scenario` wins; otherwise odd cycles select `MEMORY_EXHAUSTION` and even cycles `NETWORK_LATENCY`. The campaign narrative is logged but is not parsed to choose the fault.

Scenario strings containing `NETWORK`, `PACKET` or `LATENCY` reach the network executor. Other strings reach the resource executor, where `CPU` selects CPU load, `CRASH` selects the crash endpoint and all remaining values select memory allocation. Redis starvation exists in the catalog but is not selected by `REDIS_CONNECTION_STARVATION` in this dispatcher.

Role	Model use and decision authority
Attack planner	Claude supplies a two-sentence narrative; Python chooses the scenario.
Resource stresser	Claude explains the mechanism; Python selects a catalog method.
Network injector	Claude explains tc effects; Python selects delay or loss.
Incident monitor	Sequential health probes and fixed breach rules; no model call.
Investigator	Rules set root cause, confidence and runbook; Claude adds explanatory text.
Recovery fixer / report writer	Fixed runbook dispatch, health verification and Markdown formatting.

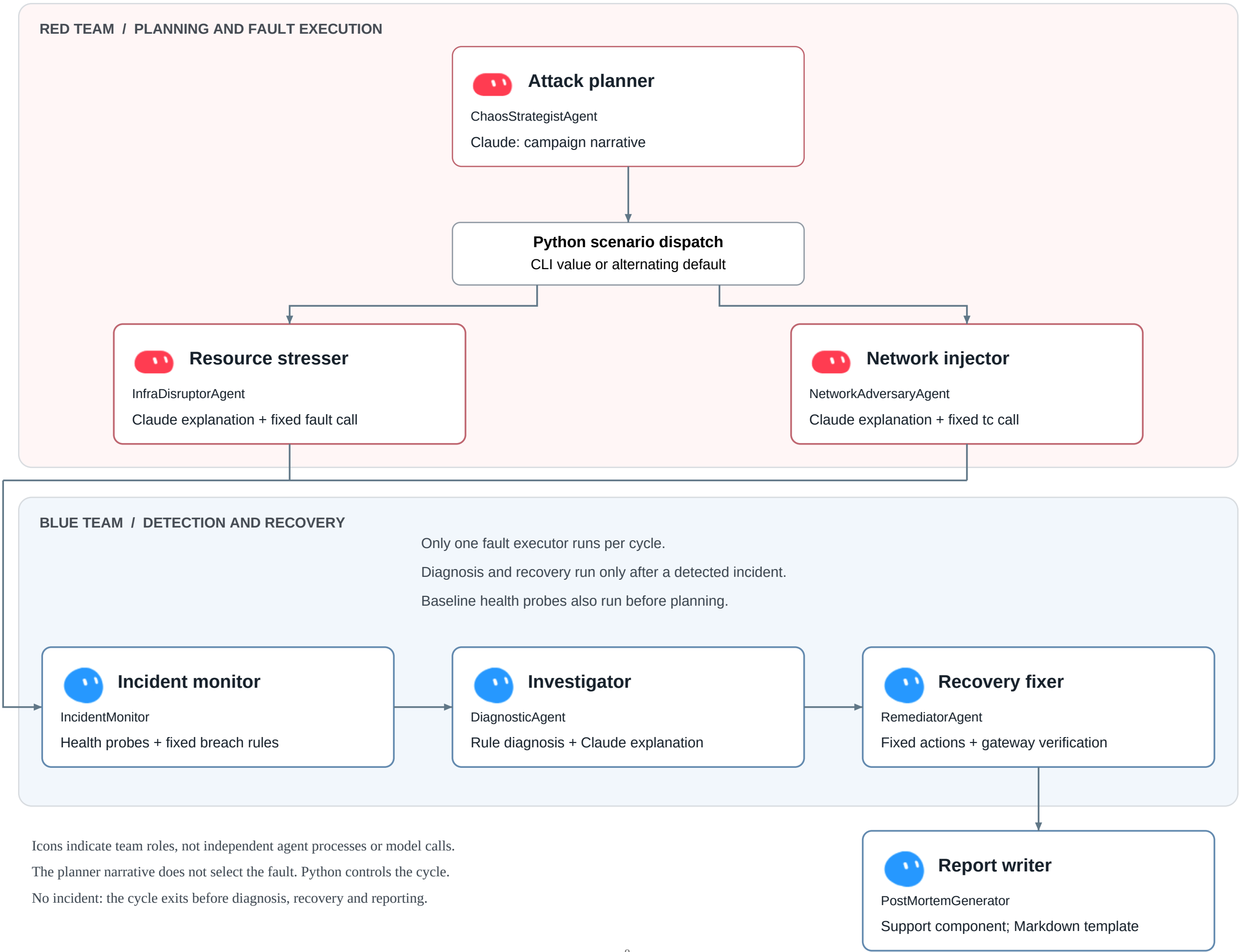
Anthropic request contract

The shared engine calls the Anthropic Messages API with a 90-second HTTP timeout and 2,048 output tokens. Ordinary requests use temperature 0.2. Investigator requests enable a 1,024-token thinking budget for Sonnet and use temperature 1.0; the engine retains only returned text blocks, not thinking blocks.

Configured defaults are `claude-sonnet-4-6` for planning, resource execution and diagnosis, and `claude-haiku-4-5-20251001` for network execution. `MODEL_*` environment variables can override mappings. Monitor, remediator and report model settings are present but unused by their primary implementations. A missing key or failed API response returns `None`, and the role continues with fallback text.

Agent architecture and cycle

Six primary team roles, plus the separate incident report writer



4.1 Agent cycle walkthrough

The preceding diagram shows the three Red Team roles, three Blue Team roles and separate report writer. One orchestrator controls their execution; the two fault executors are alternatives, not parallel steps.

1. Establish the baseline

IncidentMonitor probes the three service health endpoints before planning. The resulting status and gateway latency snapshot is passed to the planner as context.

2. Produce the campaign description

ChaosStrategistAgent asks Claude for a short narrative based on the baseline and target services. The narrative is recorded; it is not parsed into executable instructions or used to select the scenario.

3. Select one executor

Python uses --scenario when supplied; otherwise odd cycles select memory exhaustion and even cycles select network latency. Strings containing NETWORK, PACKET or LATENCY select NetworkAdversaryAgent; other values select InfraDisruptorAgent.

4. Execute the selected fault

The selected executor requests a Claude explanation and makes a fixed catalog call. The resource path selects CPU, crash or memory behavior; the network path selects delay or packet loss. The executor returns a fault-event record to the orchestrator.

5. Detect an incident or end the cycle

IncidentMonitor performs up to nine post-injection checks. A gateway probe over 1,200ms or any service status of 500 or higher declares an incident. If no incident is declared, the cycle returns before diagnosis, recovery and reporting.

6. Diagnose the incident

DiagnosticAgent applies ordered rules to service status, latency and available Kubernetes events. Rules select the root cause, confidence and runbook. Claude supplies additional explanatory text without changing those selections.

7. Run recovery and verify the gateway

RemediatorAgent applies fixed reset/restart logic, then checks gateway health up to five times. One HTTP 200 response below 800ms returns RESOLVED; otherwise it returns ESCALATED. This stage does not call an LLM.

8. Write the incident report

PostMortemGenerator receives the fault, incident, diagnosis, remediation and elapsed timing values. It writes a Markdown template under reports/. It runs after remediation for either verification outcome; Section 9 explains how its displayed status differs from the verification record.

5 Fault mechanisms and recovery endpoints

Fault	Executed mechanism
Memory allocation	Order /chaos/leak-memory retains byte buffers. The executor requests 150 MiB. OOM depends on total process usage and the actual cgroup limit.
CPU pressure	Order /chaos/cpu-burn schedules a 15-second compute task with occasional async yields.
Network delay	Payment /chaos/latency deletes the root qdisc and attempts netem delay 2500ms with 100ms jitter on eth0.
Packet loss	Payment /chaos/drop-rate attempts a 35% netem loss rule on eth0.
Redis starvation	Order /chaos/redis-starvation opens and retains clients; default request is 55 against Compose maxclients=50. Catalog/direct endpoint path.
Application delay / errors	Order endpoints configure async delay and probabilistic errors; payment outage toggles HTTP 503 responses.
Reset	Order clears retained buffers, client connections and fault flags. Payment deletes the root qdisc and clears its flags.

Execution results

Network endpoints return `tc returncode`, `stdout` and `stderr` and update the active flag based on command success. `NET_ADMIN` and `iproute2` are needed for the intended qdisc changes. Compose supplies `NET_ADMIN`; the Kubernetes target manifest does not add that capability. An HTTP response alone does not establish that a kernel rule was applied.

The crash endpoint starts a thread whose target calls `sys.exit(1)`; that exits the thread rather than guaranteeing termination of the service process. CPU tasks end on their own timer and are not tracked for cancellation by the reset endpoint. These distinctions prevent expected-impact labels from being mistaken for verified effects.

Fault evidence

The executors log `fault_type`, `target_service`, `command/result` summaries and elapsed execution time. They do not preserve all catalog return details in their emitted event, and they label status `FAULT_ACTIVE` without independently verifying the intended fault. Consult service results and metrics when assessing an individual run.

6 Detection, diagnosis and recovery

Incident detection

After injection, the orchestrator performs up to nine iterations, each with a one-second sleep followed by sequential health probes. Gateway timeout is four seconds; order and payment timeouts are three seconds each. Probe and logging time make an iteration longer than one second.

A gateway probe duration above 1,200ms or any service HTTP status of 500 or higher declares SEV-1. Although the setting and message call the duration p99, the value is one measured gateway health request. The configured 2% error-rate threshold is not evaluated in this monitor.

Diagnostic precedence

Observed condition	Rule-selected response
Payment 502/504, or latency >2000ms with order 200	DOWNSTREAM_PAYMENT_LATENCY; confidence 0.95; RESET_PAYMENT_PARAMETERS.
Order 500/502/504 with an OOM-related event	MEMORY_EXHAUSTION_OOM; 0.98; EVICT_CONTAINER_AND_PURGE_LEAK.
Order 500/502/504 without that event	ORDER_PROCESS_FAILURE; 0.90; ROLLING_RESTART_ORDER_SERVICE.
Remaining latency >1000ms	CPU_QUOTA_THROTTLING; 0.88; SCALE_WORKLOAD_REPLICAS.
Otherwise	UNKNOWN_DEGRADATION; 0.50; RESTART_WORKLOAD.

These are ordered heuristic branches with fixed confidence values. Kubernetes events, when available, and the health snapshot are also sent to Claude. Its response is recorded as explanatory text and does not replace the selected runbook.

Recovery and verification

The remediator resets fault endpoints and may restart a deployment through kubectl. Its CPU branch tests whether the runbook name contains CPU; SCALE_WORKLOAD_REPLICAS therefore falls through to generic reset/restart in this implementation. Recorded action descriptions are not substitutes for Kubernetes command results.

Verification tries gateway /health up to five times. A single HTTP 200 response below 800ms returns RESOLVED; otherwise it returns ESCALATED. It does not require ten consecutive passes or recheck all downstream services. When no incident is detected, the cycle returns without a reset or report.

7 Kubernetes sandbox and deployment definitions

The agent sandbox is part of the documented AWS execution layout. `k8s/agents/agent-sandbox.yaml` defines namespace `sre-agent-sandbox`, two deployments and an ingress/egress policy. Each deployment requests 100m CPU and 128Mi memory, with limits of 500m and 512Mi. The red container disables privilege escalation and retains a writable root filesystem.

Network rules

The policy has no allowed ingress rules. Egress permits UDP/TCP 53, TCP 443 and traffic toward namespaces labeled `platform.sre.io/role=workload`. The port-443 rule is not restricted to the Anthropic hostname. The supplied target namespace instead has `arena.sre.io/role=victim`, so the intended namespace selector requires alignment for target access.

Identity and Kubernetes actions

The red service account is defined in the sandbox and bound to a Role in `sre-target-apps` with `get/list/watch` on pods and `pods/exec`. The blue deployment references `sre-defender-sa` in the sandbox, while the base file creates and binds that account in `sre-arena-control`. These namespace-specific identities must be reconciled before treating the manifests as an end-to-end deployment.

The base defender ClusterRole covers reads and mutations of pods, events, deployments and network policies across the cluster. The Python toolkit targets `sre-target-apps` by default, checks `kubectl` client availability, and runs commands with a 15-second timeout. A successful client-version check does not confirm authorization or cluster connectivity.

Target workload definitions

The target manifest defines two replicas each for gateway, order and payment, ClusterIP services and readiness probes every five seconds. Limits are 256Mi memory per container, with 300m gateway CPU and 250m for order/payment. Redis, PostgreSQL, traffic generation and observability are not included in that file; Compose defines the fuller local service stack.

Deployment artifacts

`Dockerfile.agent-sandbox` copies the primary agent packages, reports and orchestrator and starts cycles automatically. Kubernetes sandbox pods instead wait for operator execution. Remote deployment and copy helpers contain run-specific instance IDs, paths and selectors, so their values are deployment inputs rather than portable defaults.

8 AWS infrastructure and observability

Terraform topology

Terraform creates a VPC, Internet Gateway, public route table and three subnets: 10.0.1.0/24, 10.0.10.0/24 and 10.0.20.0/24. A single EC2 instance sits in the public subnet with both security groups attached. The additional subnets do not host separately provisioned application or database instances.

Defaults are us-east-1, t3.small, an Ubuntu 22.04 amd64 image and a 25 GiB gp3 root disk. Bootstrap installs Docker, Python utilities and k3s. It does not deploy application source or workloads. The instance profile attaches AmazonSSMManagedInstanceCore; the Terraform file does not provision CloudWatch log permissions or dashboards.

The ingress group permits public TCP 22, 3000, 8000 and 9090 and unrestricted outbound traffic. This is the declared configuration, rather than a claim that SSH is closed or that dashboards are private. Terraform outputs endpoint URLs derived from the instance IP; application availability depends on subsequent deployment.

Metrics and dashboards

Prometheus scrape/evaluation intervals are one second. The provisioned Grafana dashboard refreshes every second and includes availability, throughput, 5XX rate, state, latency percentiles, ingress/error comparison, process memory and service share. Those configured intervals are not end-to-end timing guarantees.

Gateway middleware records request counts and duration histograms. Order metrics cover outcomes, processing duration, retained memory and Redis starvation clients. Payment metrics cover outcomes and processing duration. The gateway sandbox-status gauge is set to one at startup with fixed labels; it does not inspect Kubernetes isolation.

Audit records

StructuredLogger writes logs/agent_activity.jsonl, prints events, posts bounded summaries to /api/agent-events, and attempts CloudWatch put_log_events in us-east-1 against /sre/autonomous-agent-audit and audit-stream. Delivery failures are suppressed. CloudWatch groups, streams and permissions must exist for that path.

Command and result fields are supplied by callers and often describe planned or summarized actions rather than captured shell stdout/stderr. Gateway labels truncate commands to 120 characters and results to 160. CloudWatch receives the selected strings without those truncations. Separate helper scripts define dashboards; the Fluent Bit file is a ConfigMap, not a deployed collector by itself.

9 Reports, configuration and operation

Incident timing and reports

MTTD measures time from just before fault execution until declaration, including the executor model call, injection and probes. MTTR measures from declaration through diagnosis, recovery and verification. These are per-cycle elapsed values, despite the mean-time names.

PostMortemGenerator writes timestamped Markdown under reports/. It uses a template, not an LLM. The executors emit fault_type while the template reads scenario, so the report can be named UNKNOWN_FAULT. The template also states RESOLVED and successful verification unconditionally; use the remediation verification event to determine actual outcome. Intermediate timeline offsets are fixed narrative offsets rather than independently measured timestamps.

Configuration

Setting	Purpose / default
ANTHROPIC_API_KEY	Enables model requests; loaded from environment/.env.
API_GATEWAY_URL	Gateway base URL; localhost:8000 by default.
ORDER_SERVICE_URL / PAYMENT_SERVICE_URL	Separate service addresses, default localhost ports 8001 and 8002.
MODEL_*	Per-role model identifiers in agents/config.py.
--cycles / --delay	Two cycles and four-second inter-cycle delay by default.
--scenario	Forces the dispatch string instead of odd/even defaults.
--target-ip	Sets all service URLs to the same host with different ports. Use individual environment URLs for separate Kubernetes services.

Local sandbox workflow

Create .env with the model key, then run docker compose up --build -d. Compose starts the application, data services, traffic generator, observability and agent-sandbox. The sandbox image automatically runs its configured cycles. Reports are mounted to the host reports directory. Use docker compose logs agent-sandbox to inspect cycle output.

AWS workflow

Terraform init/apply provisions the host and network first. Application images, source and Kubernetes manifests then need deployment and configuration. Execute the orchestrator inside the prepared sandbox with per-service addresses. The repository's helper scripts illustrate SSM deployment and copying; adapt their run-specific identifiers before use.

The teardown scripts invoke Terraform destroy for resources tracked by the selected state. Separately created CloudWatch resources or other external assets are outside that state. Local docker compose down stops the Compose stack; adding -v also removes its managed volumes.

10 Source reference and next steps

Reviewed repository: github.com/ritvikindupuri/InfraAttack

Revision: 01ad20377462bd8462da46c574698f7a04c35388.

The review traced the primary execution path and deployment definitions. No infrastructure was provisioned, faults executed or external model calls made for this documentation.

Source	Coverage
orchestrator.py	Role construction, phase order, dispatch, detection loop and timing.
agents/config.py; common/llm_engine.py	Configuration, provider requests and role/model routing.
agents/fault_injector/	Planner, resource/network executors and fault catalog.
agents/incident_response/	Health checks, heuristic diagnosis, recovery verification and reports.
agents/common/	Structured logging and Kubernetes command helpers.
services/*/main.py	Gateway, application state, metric definitions and fault endpoints.
traffic-generator/	HTTP workload generator and supplementary k6 script.
docker-compose.yml; Dockerfile.agent-sandbox	Local stack, limits, capabilities and automatic sandbox command.
k8s/; terraform/main.tf	Sandbox/workload manifests, RBAC, policy and AWS resources.
dashboards/; deployment helpers	Prometheus/Grafana configuration and operational scripts.

Additional repository paths

arena_runner.py and agents/attacker plus agents/defender provide an alternate older workflow with different classes and schemas. FaultInjectorAgent is another catalog wrapper not imported by the main orchestrator. Dashboard builders, test_* scripts and SSM helpers support development and recorded operations; they are not all executed by the main cycle.

Next steps

Priorities are aligning sandbox identities and namespace labels; preserving fault execution results; connecting Redis dispatch; and making report status depend on verification. Recovery checks could include all services and measured Prometheus windows. Model output could be constrained to a validated plan schema if it is later intended to control actions.

The system figure is the original README overview, interpreted with the implementation details in Section 2. The agent figure covers all six primary team roles and the separate report writer. Red and blue icons indicate team membership; subtitles distinguish model-assisted roles from fixed rules and template output.